บทพูดนำเสนอ อำพร OMS (สไลด์ 20 หน้า · ประมาณ 16 นาที)

ผู้พูดที่แนะนำ: **ตรัยรัตน์ (หลังบ้าน)** พูดส่วนที่เกี่ยวกับปัญหา ความต้องการ ข้อมูล สถาปัตยกรรม และการทดสอบ · **พนาวุฒน์ (หน้าบ้าน)** พูดส่วน use case หน้าจอ และ component (สไลด์ 8–10) แบ่งใหม่ได้ตามที่ถนัด

เวลาในวงเล็บคือเวลาโดยประมาณของแต่ละหน้า รวมประมาณ 16 นาที ถ้าเกินเวลาให้พูดหน้า 13 และ 14 สั้นลง ตัวหนาคือคำที่ควรเน้น

สไลด์ 1 · ปก (30 วินาที) — ตรัยรัตน์

สวัสดีครับอาจารย์ทุกท่าน วันนี้พวกผมจะนำเสนอปริญญานิพนธ์เรื่อง **ระบบจัดการคำสั่งซื้อและตรวจสอบคลังสินค้า** กรณีศึกษาร้านวัสดุก่อสร้างอำพรคอนกรีตครับ

ผมตรัยรัตน์ ทำฝั่งหลังบ้าน ส่วนพนาวุฒน์ทำฝั่งหน้าบ้านครับ

สไลด์ 2 · ปัญหาแยกตามผู้มีส่วนได้ส่วนเสีย (1 นาที 30 วินาที) — ตรัยรัตน์

เริ่มจากปัญหาครับ พวกผมไปดูการทำงานจริงที่ร้าน สัมภาษณ์เจ้าของร้าน และถามพนักงาน แล้วแยกปัญหาตามคนที่เจอ ได้ **4 กลุ่ม**

พนักงานขาย ต้องเขียนบิลกระดาษและคิดยอดเอง ผิดได้ แล้วหน้าร้านกับคลังอยู่คนละที่ จะบอกคลังทันทีก็ไม่ได้

พนักงานคลัง อ่านบิลด้วยตา หยิบของผิดชนิดผิดจำนวน มีคนเอาบิลเก่ามารับของซ้ำ และยอดสต็อกในสมุดไม่ตรงของจริง

ผู้จัดการ ข้อมูลขายกระจัดกระจาย ตรวจย้อนหลังยาก ต้องรอสรุปยอดตอนปิดร้าน และสั่งของจากความรู้สึก

ลูกค้า ไม่ได้ใช้ระบบเอง แต่เป็นคนที่รอนานที่สุด เพราะต้องถือใบเสร็จเดินไปโกดังเอง

ปัญหาทั้งหมดนี้คือหลักฐาน **EV-01 ถึง EV-09** ในเล่ม และทุกข้อจะถูกตามไปจนถึงตอนทดสอบครับ

สไลด์ 3 · วัตถุประสงค์ (45 วินาที) — ตรัยรัตน์

จากปัญหา ผมตั้งวัตถุประสงค์ 6 ข้อ สี่ข้อแรกวัดผลได้

ข้อ 1 ยอดเงินทุกใบระบบคิดให้ ข้อ 2 คำสั่งซื้อที่จ่ายเงินแล้วต้องถึงจอคลังภายใน **3 วินาที** ข้อ 3 ใบเดียวจ่ายของได้ครั้งเดียว แม้สองคนกดพร้อมกัน ข้อ 4 ผู้จัดการเห็นยอดขาย สต็อก และคิวระหว่างวัน

อีกสองข้อเป็นเชิงการเรียนรู้ครับ

สไลด์ 4 · ขอบเขต (45 วินาที) — ตรัยรัตน์

ขอบเขตคือผู้ใช้ 3 บทบาท 11 use case 4 หน่วยติดตั้ง และใช้สินค้าจริงของร้าน 783 รายการ

ฝั่งซ้ายคือสิ่งที่ทำ ตั้งแต่ขาย ส่งเข้าคิว จ่ายของ ตัดสต็อก จนถึงแดชบอร์ด ฝั่งขวาคือสิ่งที่ไม่ทำ เช่น เชื่อมธนาคาร หรือระบบบัญชีเต็มรูปแบบ

พวกผมเลือกทำเส้นทางหลักของร้านให้ครบและลึกจริงครับ

สไลด์ 5 · เก็บความต้องการอย่างไร (40 วินาที) — ตรัยรัตน์

ความต้องการเก็บมาสี่วิธีครับ สังเกตการทำงานจริงที่หน้าร้านกับคลัง สัมภาษณ์เจ้าของร้าน ถามพนักงาน และดูเอกสารเดิม อย่างบิลกระดาษกับไฟล์สินค้า

คอลัมน์ขวาบอกว่าแต่ละวิธีได้อะไรมา แล้วทั้งหมดไหลต่อกันตามแถวล่าง คือหลักฐาน 9 ข้อ ไปเป็นความต้องการ use case หน้าจอ ตาราง จนถึงกรณีทดสอบครับ

สไลด์ 6 · Iterative Waterfall และเหตุผลที่เลือก (1 นาที) — ตรัยรัตน์

งานแบ่งเป็น **6 ระยะ ระยะละประมาณหนึ่งเดือน** ตามตารางครับ วิเคราะห์ ออกแบบ พัฒนาสองระยะ ทำแดชบอร์ดกับแจ้งเตือน แล้วทดสอบ แต่ละระยะมีหมุดหมาย M1 ถึง M6 เป็นผลงานที่ส่งได้

ทำไมเลือก **Iterative Waterfall** มีสามเหตุผลครับ

หนึ่ง ขอบเขตชัดตั้งแต่ต้น ผู้ใช้มีไม่กี่คน ไม่ต้องเปลี่ยนความต้องการบ่อยแบบ Scrum

สอง งานต้องทำตามลำดับ ระยะ 4 ยืนยันจ่ายของ ต้องมีคำสั่งซื้อจากระยะ 3 ก่อน

สาม ความเสี่ยงหลักคือข้อมูลถูกต้องตอนหลายคนกดพร้อมกัน ต้องออกแบบให้ชัดก่อนเขียนโค้ด

คำว่าทวนกลับได้ คือย้อนกลับไปแก้ระยะก่อนได้ เราย้อนจริงสองครั้ง คือ DF-01 เรื่องเลขที่ชนกัน กับ DF-03 เรื่องวิธีพิมพ์บน Windows ครับ

สไลด์ 7 · ความต้องการมาจากใคร และ System Requirements (1 นาที) — ตรัยรัตน์

จากปัญหาเราได้ functional requirement 9 ข้อ ตารางนี้บอกว่าแต่ละข้อ **มาจากใคร**

เช่น FR-03 สร้างคำสั่งซื้อและพิมพ์ใบเสร็จ มาจากพนักงานขาย FR-06 ตัดสต็อกและกันจ่ายซ้ำ มาจากพนักงานคลัง ส่วน FR-07 แดชบอร์ด มาจากผู้จัดการ

ฝั่งขวาคือ **system requirement** สิ่งที่ระบบต้องต่อหรือต้องทำได้ เช่น ต่อ PromptPay QR ต่อเครื่องพิมพ์ความร้อนให้พิมพ์ภาษาไทยภายใน 5 วินาที ใช้ WebSocket สำรองด้วย SSE และต้องติดตั้งบนคอมเครื่องเดียวของร้านได้โดยไม่ต้องต่ออินเทอร์เน็ตครับ

สไลด์ 8 · Use Case Diagram (1 นาที) — พนาวุฒน์

ต่อจากความต้องการ เราแตกออกมาเป็น **11 use case** ครับ กรอบสี่เหลี่ยมคือขอบเขตระบบ คนอยู่นอกกรอบ

พนักงานขายซ้ายบน ทำ UC-03 สร้างคำสั่งซื้อ ซึ่ง include UC-04 ออกเลขที่ไม่ซ้ำทุกครั้ง พนักงานคลังซ้ายล่าง ดูคิว ปรับสถานะ และ UC-07 ยืนยันจ่าย ผู้จัดการขวาบน จัดการสินค้า ดูแดชบอร์ด และค้นประวัติ

ขวาล่างคือ**ระบบอัตโนมัติ** ไม่ใช่คน เป็นตัวจับเวลาที่คอยตรวจคิวค้าง แล้วแจ้งเตือนเองครับ

ผู้จัดการทำได้ทุกอย่างที่พนักงานขายและคลังทำด้วยครับ

สไลด์ 9 · ความต้องการ → Use Case → หน้าจอ (1 นาที) — พนาวุฒน์

หน้านี้คือการไล่จากความต้องการไปถึงหน้าจอครับ

ความต้องการหนึ่งข้อแตกได้หลาย use case เช่น FR-04 คิวเรียลไทม์ แตกเป็น UC-05 ดูคิว กับ UC-06 ปรับสถานะ

แล้วหน้าจอหนึ่งหน้าก็รวมได้หลาย use case เช่น **หน้าขาย FE-02** ทำทั้ง UC-03 สร้างคำสั่งซื้อ และ UC-04 ออกเลขที่ไม่ซ้ำ

สรุปคือ 9 ความต้องการ 11 use case ออกมาเป็น 9 หน้าจอครับ

สไลด์ 10 · หน้าจอ → Component → API → ตาราง (1 นาที) — พนาวุฒน์

ยกหน้าขายเป็นตัวอย่างครับ หนึ่ง page ประกอบด้วย component ย่อย

เมนูซ้ายกับปุ่มออกจากระบบคือ Layout ที่ทุกหน้าใช้ร่วมกัน และมี RequireRole คอยกันคนที่บทบาทไม่ตรง

ช่องค้นหาเรียก GET /products ไปอ่านตาราง products ตารางรายการที่เลือกเรียก /orders/quote ให้หลังบ้านคิดราคาจากฐานข้อมูล ไม่เชื่อราคาจากหน้าจอ

พอกดยืนยันรับชำระ จะเรียก POST /orders ลงตาราง orders, order_items และ audit_logs

ส่วนจอคลังใช้ events.js ฟังข้อมูลสด ไม่ต้องรีเฟรชครับ

สไลด์ 11 · ข้อมูลที่แต่ละ Use Case ใช้ (สร้าง อ่าน แก้ไข) (1 นาที) — ตรัยรัตน์

ก่อนออกแบบฐานข้อมูล เราไล่ทีละ use case ว่าต้อง**สร้าง อ่าน หรือแก้ไข**ข้อมูลอะไรครับ สีเขียวคือสร้าง สีน้ำเงินคืออ่าน สีส้มคือแก้ไข

เช่น UC-03 สร้างคำสั่งซื้อ อ่านสินค้าเพื่อเอาราคา แล้วสร้างคำสั่งซื้อกับรายการสินค้า

จุดสำคัญคือ **UC-07 ยืนยันจ่าย** เป็นกรณีเดียวที่ทั้งอ่านและแก้ไขสินค้ากับคำสั่งซื้อพร้อมกัน เลยต้องออกแบบ transaction ละเอียดที่สุด

จากตารางนี้เราค่อยรวมเป็นฐานข้อมูลเดียวในหน้าถัดไปครับ

สไลด์ 12 · ER Diagram (1 นาที) — ตรัยรัตน์

ได้ฐานข้อมูล **7 ตาราง** ครับ users, products, orders, order_items, stock_movements, notifications และ audit_logs

มีสองจุดที่ตั้งใจออกแบบ

หนึ่ง order_items เก็บ**ราคา ณ เวลาขาย** ถ้าวันหลังแก้ราคาสินค้า ใบเก่าก็ไม่เปลี่ยน

สอง stock_movements เก็บ**ทุกครั้งที่สต็อกเปลี่ยน** พร้อมยอดหลังเปลี่ยนและสาเหตุ ถ้าของหาย จะตามได้ว่าหายเพราะอะไร ซึ่งตอบปัญหาสต็อกไม่ตรงของพนักงานคลังครับ

เส้นที่มีเลข 1 กับ N คือหนึ่งแถวโยงได้หลายแถว เช่นคำสั่งซื้อหนึ่งใบมีได้หลายรายการสินค้าครับ

สไลด์ 13 · C1 แผนภาพบริบท (30 วินาที) — ตรัยรัตน์

ระดับ C1 ระบบอยู่ตรงกลาง ผู้ใช้ 3 บทบาทใช้ผ่านเบราว์เซอร์ ภายนอกมี PromptPay QR กับเครื่องพิมพ์ และ**ไม่เชื่อมธนาคาร** พนักงานขายเป็นคนดูว่าเงินเข้าแล้ว

สไลด์ 14 · C2 แผนภาพคอนเทนเนอร์ (45 วินาที) — ตรัยรัตน์

ระดับ C2 แตกระบบเป็น **4 หน่วยติดตั้ง** ในกรอบเส้นประครับ Frontend, Backend, Database และ Print Service

Frontend เรียก Backend ด้วย REST API ส่วนข้อมูลสดอย่างคำสั่งซื้อใหม่ Backend เป็นฝ่ายผลักกลับไปผ่าน WebSocket ถ้าต่อไม่ได้ถอยไปใช้ SSE

นอกกรอบคือ PromptPay QR Library กับเครื่องพิมพ์ความร้อนครับ

สไลด์ 15 · ทำไมเลือกเทคโนโลยีเหล่านี้ (1 นาที) — ตรัยรัตน์

เหตุผลของแต่ละหน่วยครับ

React เพราะอัปเดตเฉพาะส่วนที่เปลี่ยนตอนข้อมูลผลักเข้ามา และใช้บนมือถือในคลังได้ Node.js เพราะค้างการเชื่อมต่อกับจอคิวหลายเครื่องได้ MySQL InnoDB เพราะมี transaction กับ row lock ที่การตัดสต็อกต้องใช้

บริการพิมพ์แยกออกมาเพราะต้องอยู่เครื่องที่ต่อเครื่องพิมพ์ และถ้าพิมพ์ล่ม การขายต้องยังสำเร็จ

ข้อมูลสดใช้ WebSocket สำรองด้วย SSE และถ้าสายหลุดแล้วต่อกลับ จอคิวจะโหลดใหม่ทั้งหมด คิวไม่ตกหล่นครับ

สไลด์ 16 · C3 Component (45 วินาที) — ตรัยรัตน์

ระดับ C3 คือข้างในหลังบ้านครับ ทุกคำขอต้องผ่าน **SP-01** ตรวจโทเคนและบทบาทก่อน แล้วถึงเข้าโมดูล BE-01 ถึง BE-10

โมดูลที่สำคัญที่สุดคือ BE-05 ยืนยันจ่าย ที่เรียก BE-06 ตัดสต็อก ทั้งหมดอยู่ใน transaction เดียว สำเร็จก็ commit ผิดตรงไหนก็ rollback ทั้งใบครับ

สไลด์ 17 · การทดสอบ (1 นาที 15 วินาที) — ตรัยรัตน์

การทดสอบมี 4 ระดับครับ

Unit กับ Integration ใช้ Jest กับ Supertest 14 ชุด บนฐานข้อมูลทดสอบที่แยกออกมา ผ่าน **134 จาก 134**

System ใช้ Playwright เปิดเบราว์เซอร์จริง 13 กรณี เช่นเปิดสองจอ ขายจอหนึ่งแล้วดูอีกจอว่าคิวขึ้นเอง ผ่าน **13 จาก 13**

NFR วัดจริง คิวถึงจอคลังสูงสุด **80 มิลลิวินาที** เกณฑ์คือ 3 วินาที พิมพ์ใบเสร็จประมาณ **2.3 วินาที**

กรณีที่สำคัญที่สุดคือ **TC-06** สองคนกดจ่ายใบเดียวกันพร้อมกัน ต้องสำเร็จครั้งเดียวและหักสต็อกครั้งเดียว ถ้าอาจารย์อยากดู ผมรันให้ดูได้เลยครับ

ส่วน UAT ยังรอผลครับ เดี๋ยวอธิบายหน้าถัดไป

สไลด์ 18 · UAT : ตั้ง Scenario จากปัญหา (1 นาที 15 วินาที) — ตรัยรัตน์

UAT เราตั้ง scenario **จากปัญหาของแต่ละคน** ที่เล่าไปในหน้าแรก เพื่อพิสูจน์ว่าปัญหานั้นหายไปจริงไหม

เช่นปัญหาลูกค้าต้องเดินถือบิลไปโกดัง scenario คือขายที่หน้าร้านแล้วไปดูจอคลังในโกดัง ต้องเห็นรายการขึ้นเอง และคลังจัดของได้ก่อนลูกค้ามาถึง

ปัญหาบิลเก่ามารับซ้ำ scenario คือเอาเลขที่ใบที่จ่ายแล้วมาขอรับอีก ระบบต้องไม่ให้จ่าย

ปัญหาสต็อกไม่ตรง ครบสัปดาห์ให้นับของจริงเทียบกับยอดในระบบ

แผนคือให้พนักงานจริงทั้งสามบทบาท**ใช้งานจริง 1 สัปดาห์** ทำครบทุก scenario เก็บภาพเป็นหลักฐาน แล้วค่อยประเมินด้วยแบบประเมิน 14 ข้อ

ตอนนี้**ยังไม่มีผล** เพราะต้องใช้งานต่อเนื่องให้ครบสัปดาห์ก่อน ผลจะบันทึกในตารางที่ 4.10 ครับ

สไลด์ 19 · สรุป ข้อจำกัด ข้อเสนอแนะ (1 นาที) — ตรัยรัตน์

สรุปครับ ระบบพัฒนาครบ 11 use case 4 หน่วยติดตั้ง ผ่าน Unit และ System ครบ ส่วน UAT รอผล

ข้อจำกัด เช่น ทดสอบบนคอมเครื่องเดียว ยังไม่ได้ทดสอบกับผู้ใช้จริง พนักงานยังเป็นคนยืนยันว่าเงินเข้า การเชื่อมต่อยังไม่เข้ารหัส และยังไม่มี backup อัตโนมัติ

ข้อเสนอแนะ เช่น ตรวจยอดโอนอัตโนมัติ สแกนบาร์โค้ด เปิด HTTPS และ backup ฐานข้อมูลทุกวันครับ

สไลด์ 20 · ขอบคุณ (15 วินาที) — ทั้งคู่

ขอบคุณครับ ต่อไปพวกผมจะเดโมระบบจริงให้ดู และพร้อมตอบคำถามครับ

ลำดับเดโมหลังนำเสนอ (แนะนำ)

1. เข้าสู่ระบบเป็นพนักงานขาย ขายสามรายการ แสดง QR แล้วยืนยันรับชำระ (S-05, S-06)
2. ให้อีกคนถือมือถือเปิดหน้าคิวไว้ ให้อาจารย์เห็นรายการขึ้นเองโดยไม่ต้องรีเฟรช (S-07) **จุดเด่นที่อาจารย์ย้ำ**
3. คลังเปิดใบ ยืนยันจ่าย แล้วลองจ่ายใบเดิมซ้ำให้เห็นว่าระบบไม่ยอม (S-10, S-11)
4. ผู้จัดการเปิดแดชบอร์ด ดูแจ้งเตือนของใกล้หมด และค้นประวัติ (S-13, S-15, S-16)
5. ถ้าอาจารย์ขอดูเทส: `npx jest backend/tests/integration/dispatch.test.js`

ประโยคที่ห้ามพูด

- ห้ามพูดว่าผู้ใช้ยอมรับระบบแล้ว หรือพูดตัวเลข 4.72 เพราะตอนนี้ UAT ยังไม่มีผล
- ห้ามพูดว่าเอาต้นแบบหน้าจอไปให้ร้านดูก่อน ถ้าไม่ได้ทำจริง